

人工神經網路入門

第 11 週 | 從 Perceptron 到 MLP，走完一次前向與反向傳播

後續八週課程 | 每週 180 分鐘（含兩次 10 分鐘休息）

本週成果與節奏

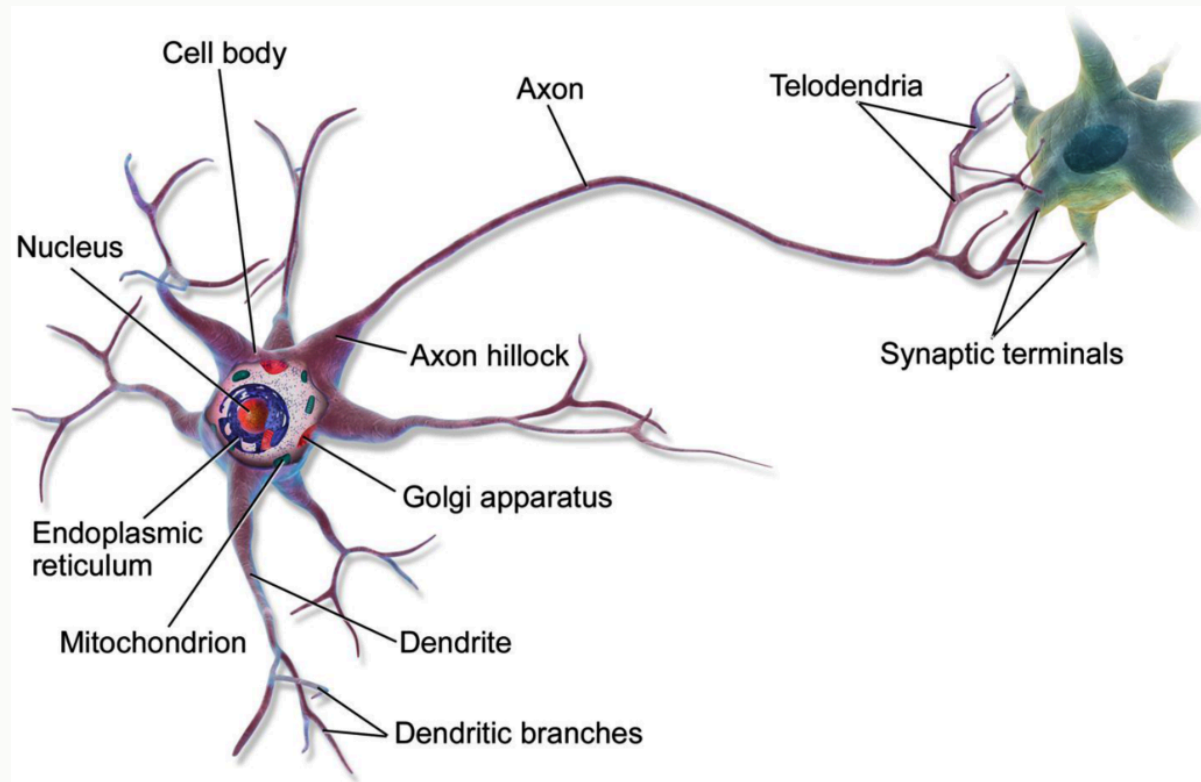
0–50 分：人工神經元、Perceptron、XOR 與 MLP。

60–120 分：非線性、鏈式法則與反向傳播。

130–180 分：迴歸／分類架構、訓練與調參。

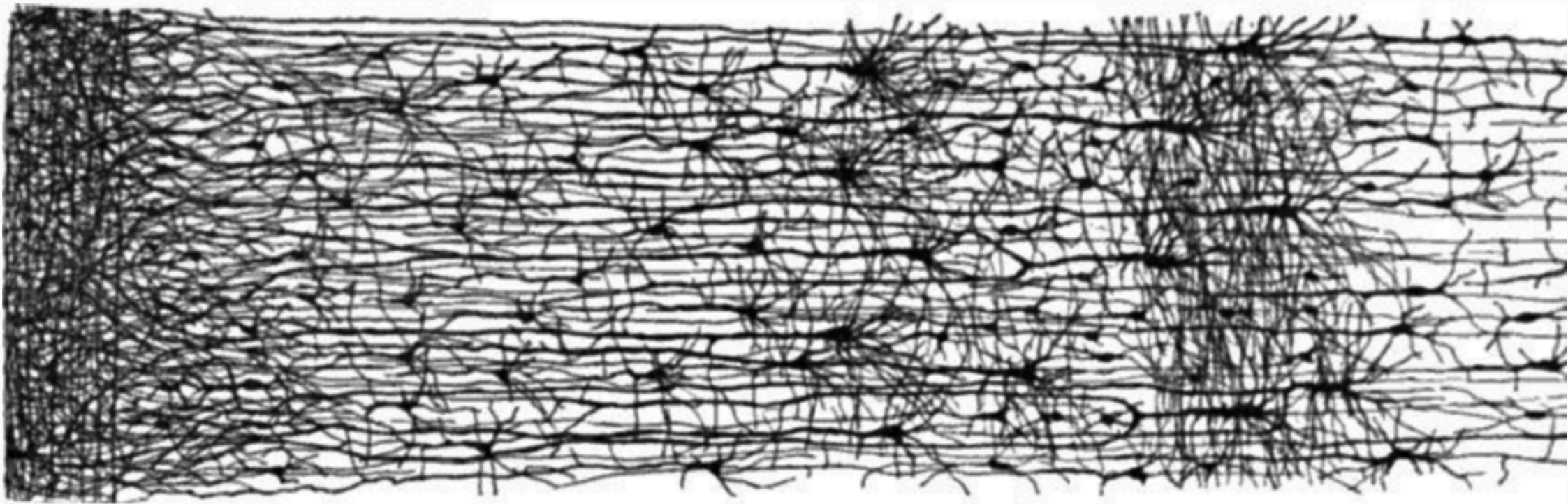
成果：能列出網路形狀、手算梯度，完成小型 MLP 比較。

從生物神經元得到啟發



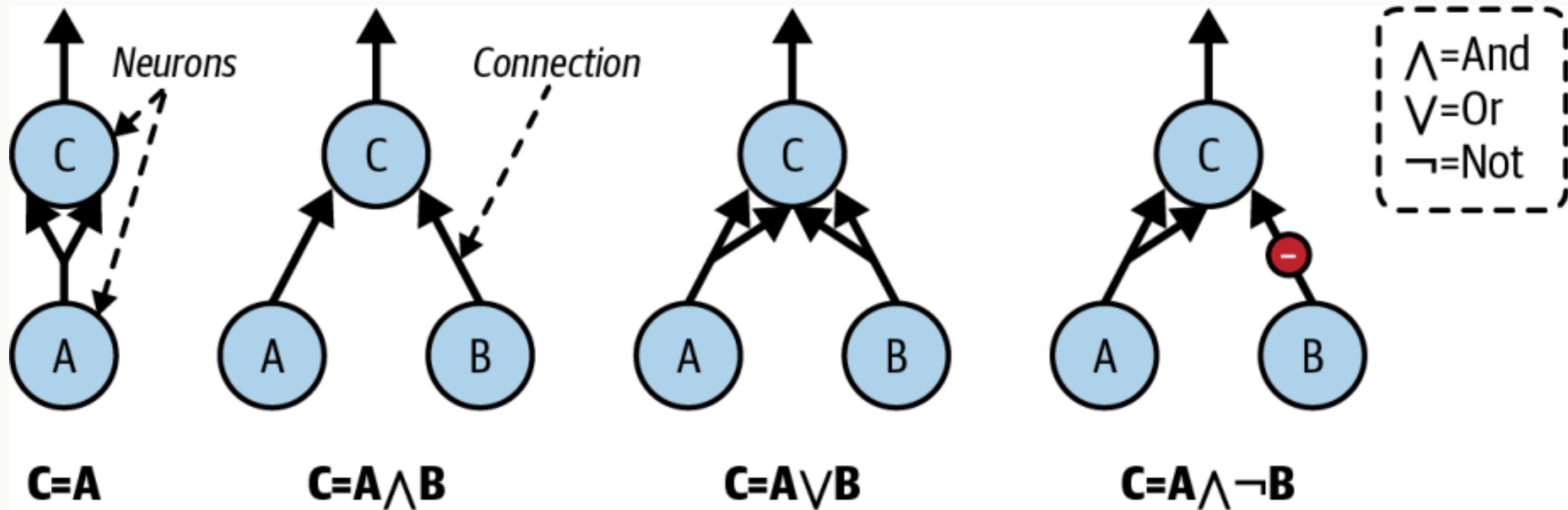
人工神經元是數學模型；樹突、細胞體與軸突提供概念類比，不等於完整腦科學模擬。

很多單元組合，形成更複雜的處理



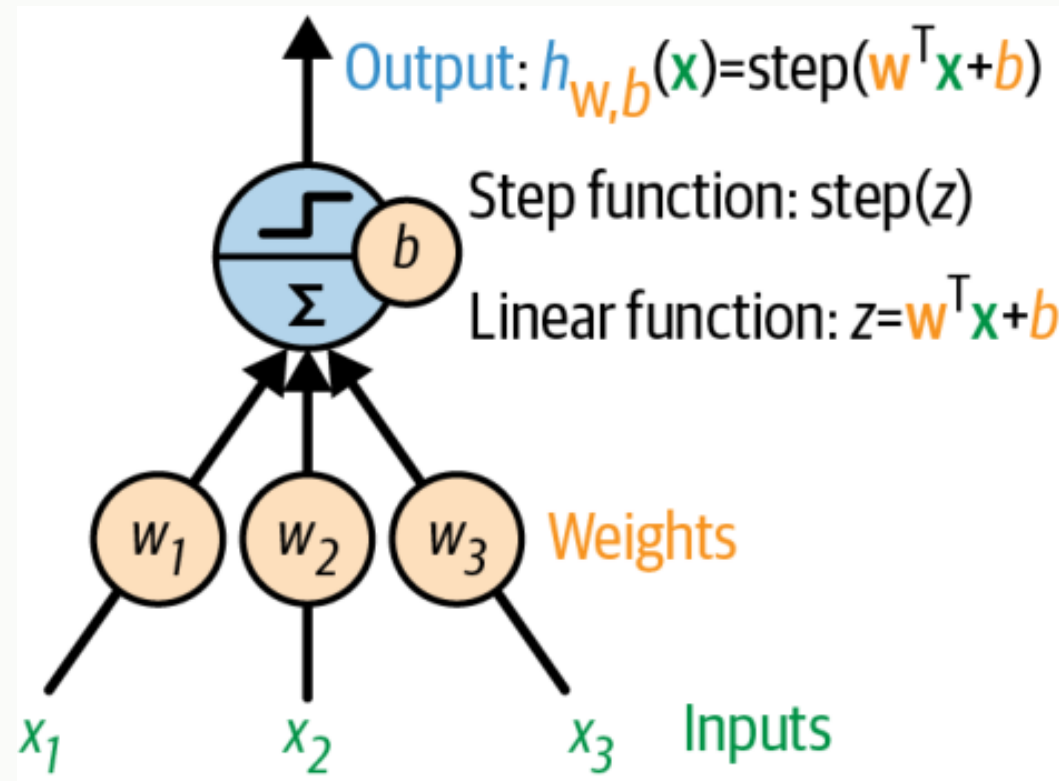
連接方式與權重決定訊息如何傳遞；學習是根據資料調整這些參數。

人工神經元可組合簡單邏輯



AND、OR、NOT 等邏輯可由不同連接與閾值表達；先觀察輸入如何決定輸出。

TLU：加權和，加偏置，再套階躍函數



輸入可以是實數；使用階躍時，輸出仍是離散的 0/1 或 $-1/+1$ 。

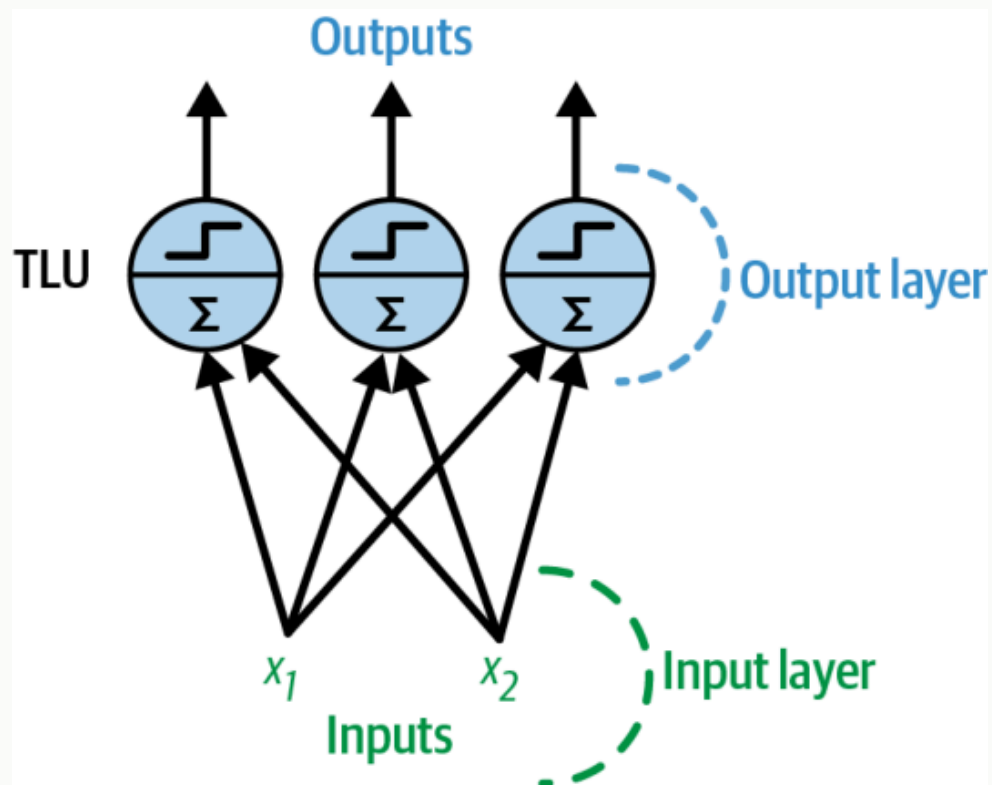
階躍函數與單層輸出

$$\text{step}(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}, \quad \text{sgn}(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$$

$$f(X) = \phi(XW + b) \circ$$

$X : m \times n$, $W : n \times q$, $b : q$; 偏置廣播到每列，輸出為 $m \times q$ 。

Perceptron 層：每個輸出都看所有輸入



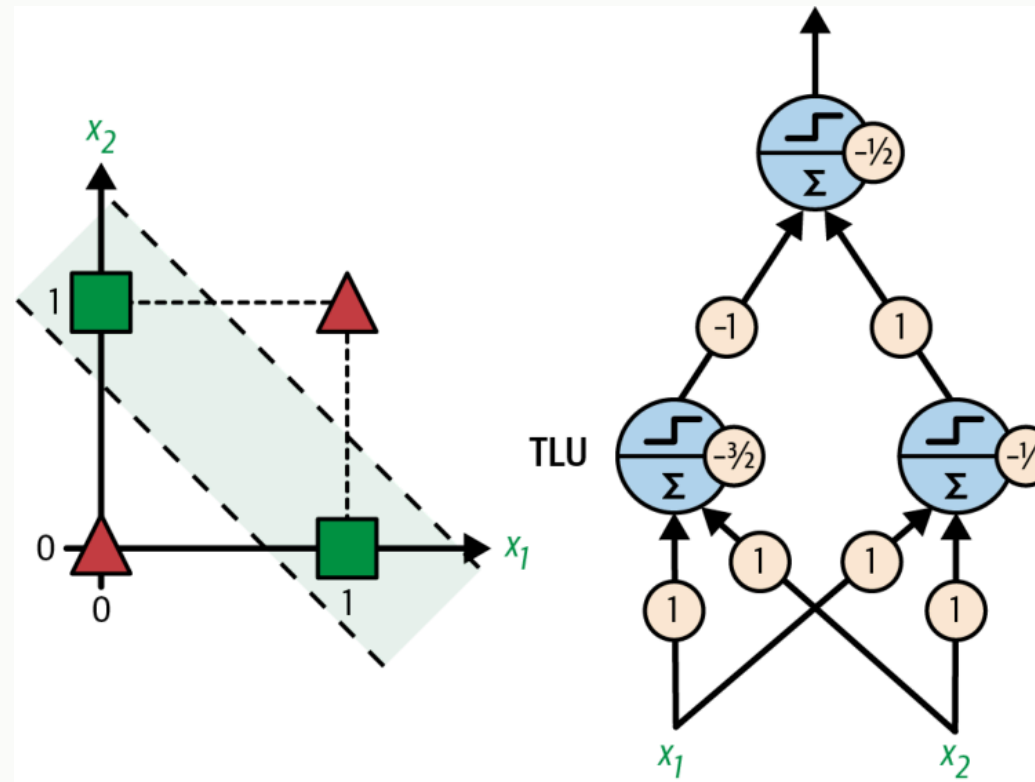
全連接層有一組輸入到輸出的權重；偏置可視為固定輸入 1 的连接。

Perceptron 學習規則：答錯才修正

$$w_{i,j} \leftarrow w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

偏置同樣更新，等價令對應輸入為 1。線性可分資料可收斂到分隔面，但不保證最大間隔，也不直接輸出機率。

XOR：一條線不夠，但組合多個單元可以



相同輸入位元輸出 0；不同輸出 1。隱藏層先建立中間邏輯特徵。

XOR 真值表：把每一列實際走過網路

A	B	A OR B	A AND B	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

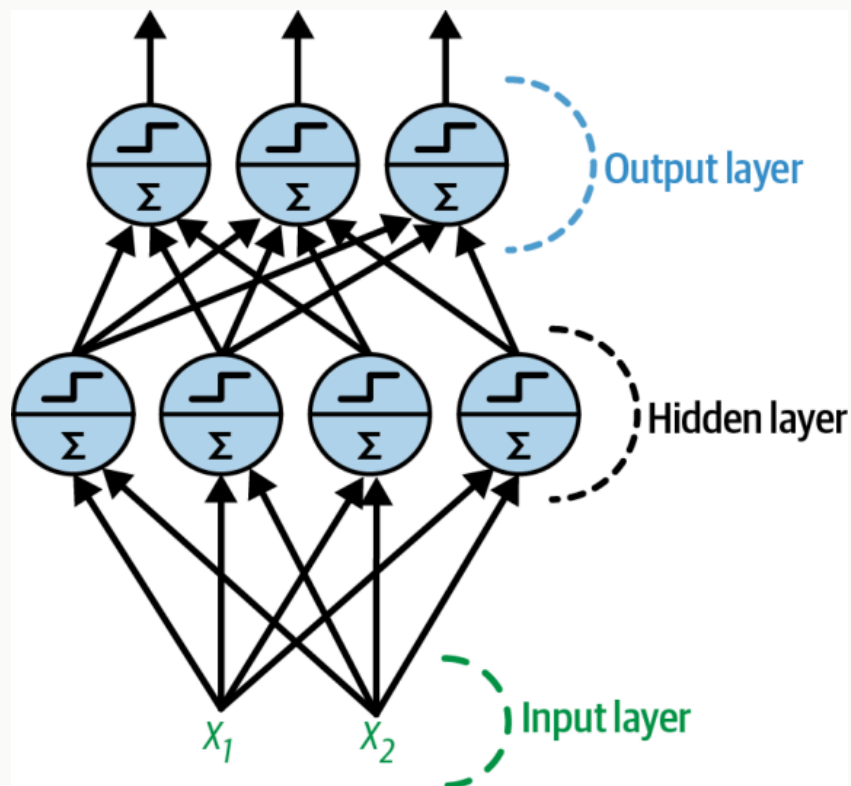
手算 XOR 網路的四組輸入

$$h_1 = \text{step}(A + B - 1.5), h_2 = \text{step}(A + B - .5)。$$

$$\hat{y} = \text{step}(-h_1 + h_2 - .5)。$$

代入四組輸入，確認輸出與真值表一致。再說明隱藏層增加了什麼能力。

MLP：輸入、隱藏層與輸出層



前饋網路的訊號向前流動；每層產生下一層的特徵表示。

休息 10 分鐘

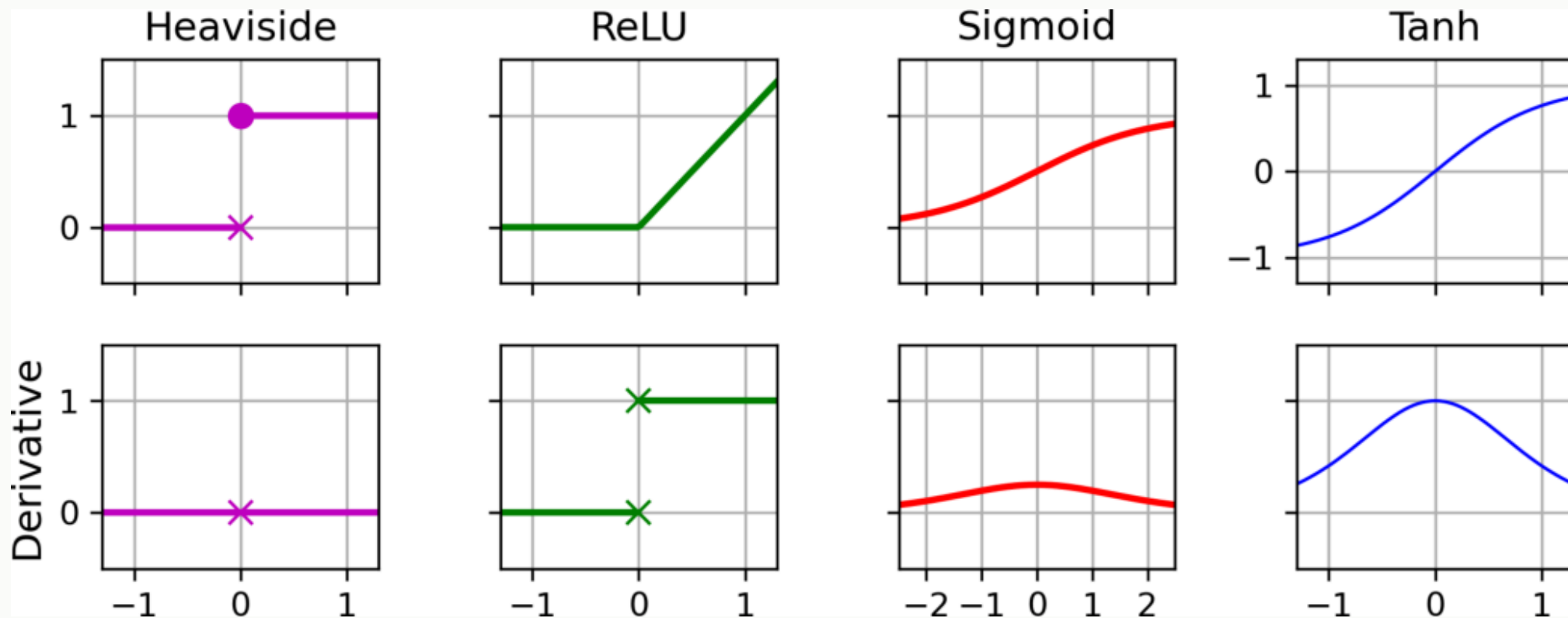
離開座位、休息眼睛。回來後先用一句話回答上一段的核心問題。

沒有非線性，堆再多層也會折成一層

$$f(g(x)) = (xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$$

因此需要非線性激活函數，才能讓深度帶來新的表達能力。

Sigmoid、tanh、ReLU 與它們的導數



Sigmoid／tanh 的飽和區梯度很小；ReLU 的負半軸梯度為零。

激活函數：用公式連結曲線與梯度

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}(z) = \max(0, z)$$

ReLU 在 $z > 0$ 導數為 1， $z < 0$ 為 0；零點採實作約定。

一次訓練步驟要分清楚四件事

1. 前向：計算各層輸出，保存中間值。
2. 損失：比較預測與真值。
3. 反向：用鏈式法則計算每個參數的梯度。
4. 更新：optimizer 根據梯度調整參數。

鏈式法則：沿路相乘，分支相加

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}$$

$z = wx + b$ 、 $a = \sigma(z)$ 。先計算靠近損失的梯度，再往前傳。

一個參數若影響多條路徑，將各路徑貢獻相加。

手算範例：一個 Sigmoid 神經元

$$x = 2, w = .5, b = 0, y = 1, L = \frac{1}{2}(a - y)^2 \circ$$

$$z = 1, \quad a = \sigma(1) \approx .731059, \quad L \approx .036165$$

$$\frac{\partial L}{\partial z} = (a - y)a(1 - a) \approx -.052877$$

接續手算：把梯度傳給權重與偏置

沿用 $\partial L / \partial z \approx -.052877$ 、 $x = 2$ 。

1. 求 $\partial L / \partial w$ 、 $\partial L / \partial b$ 。
2. 用學習率 0.1 更新 w, b 。
3. 重算預測，判斷損失是否下降。

多層網路的形狀與反向遞迴

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}, \quad A^{(l)} = \phi_l(Z^{(l)})$$

$$\delta^{(l)} = (\delta^{(l+1)}W^{(l+1)T}) \odot \phi_l'(Z^{(l)})$$

$$\nabla_{W^{(l)}} L = A^{(l-1)T} \delta^{(l)}$$

本頁 δ 定義為平均損失對 Z 的梯度，已包含批次平均因子。

初始化：打破隱藏神經元的對稱

同層神經元若具有完全一樣的權重與偏置，收到相同梯度後仍會一樣。

隨機初始化權重讓它們學不同表示；偏置常可設為 0。

多層小導數相乘會讓梯度消失；初始化尺度、激活與網路深度需一起考慮。

休息 10 分鐘

離開座位、休息眼睛。回來後先用一句話回答上一段的核心問題。

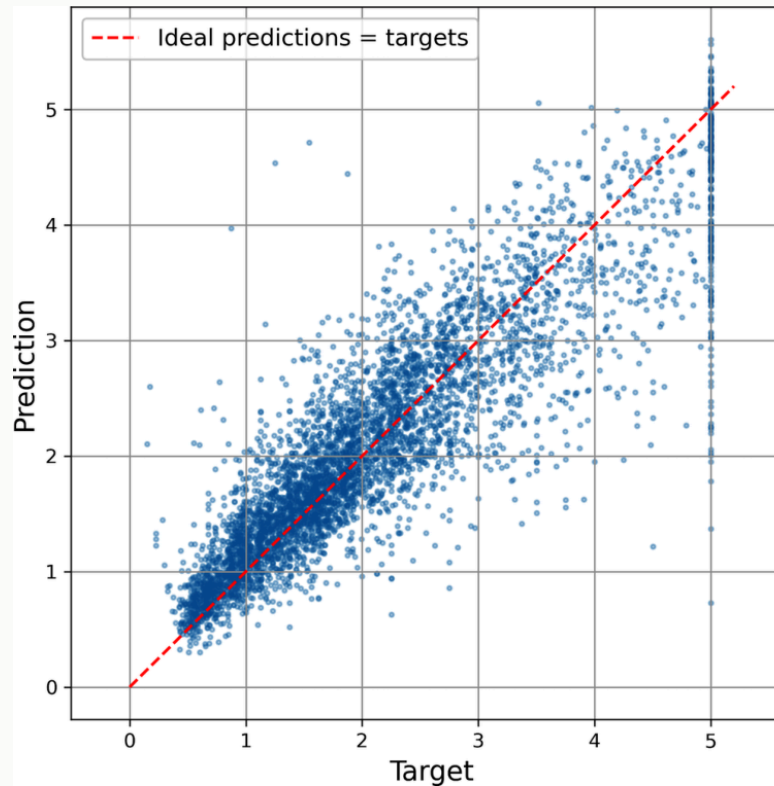
表 9-1：迴歸 MLP 的典型架構 (1/2)

項目	教材典型值
隱藏層數	依問題，常見 1–5
每隱藏層神經元	依問題，常見 10–100
輸出神經元	每個目標維度 1 個
隱藏層激活	ReLU

表 9-1：輸出範圍與損失 (2/2)

需求	輸出激活／損失
任意實數	線性輸出
非負／正值	ReLU／softplus
有界範圍	Sigmoid／tanh，配合目標縮放
迴歸損失	MSE；有離群值可考慮 Huber

MLP 迴歸：預測與真值散點



越接近對角線越好；還要看高目標值、殘差與單位。教材 RMSE 約 0.53 是該次實驗結果。

用 scikit-learn 建立小型迴歸 MLP

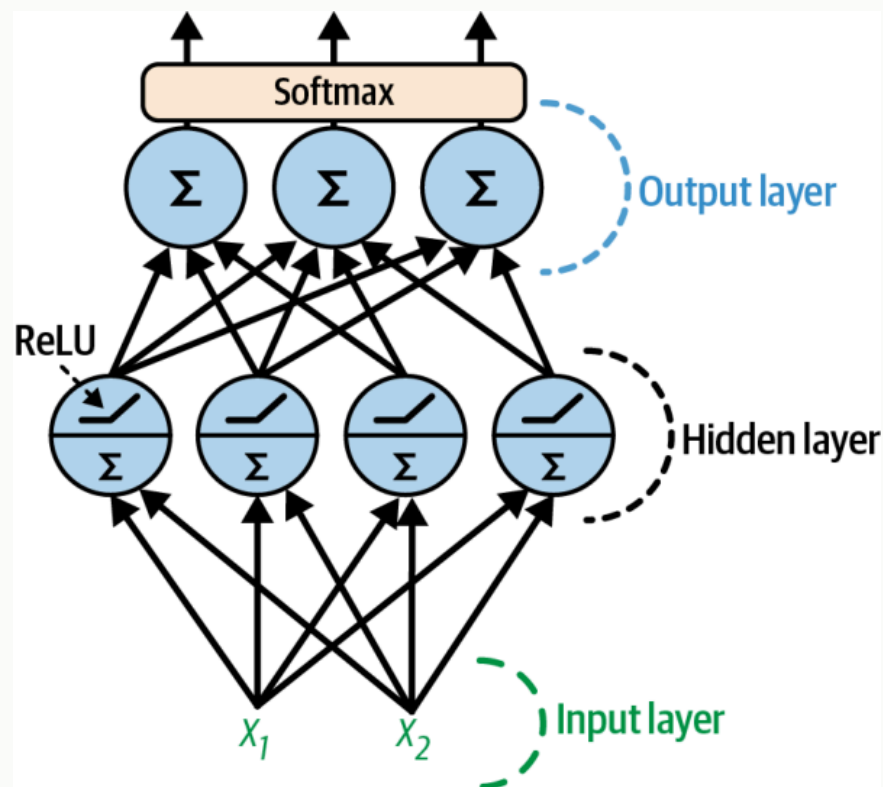
```
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
reg = make_pipeline(StandardScaler(), MLPRegressor(
    hidden_layer_sizes=(50, 50), early_stopping=True,
    max_iter=300, random_state=42))
reg.fit(X_train, y_train)
```

表 9-2：分類的輸出層與損失

共通設定：隱藏層通常 1-5 層，依任務調整。

任務	輸出個數	激活	損失
二元	1	Sigmoid	二元交叉熵
多標籤	每標籤 1	各自 Sigmoid	各標籤二元交叉熵
互斥多類別	每類 1	Softmax	多類別交叉熵

分類 MLP：隱藏層 ReLU，輸出層 Softmax



對互斥多類別產生總和為 1 的機率；這些機率仍可能過度自信。

Fashion MNIST：由像素到服飾類別



每張 28×28 灰階圖可攤成 784 個特徵；10 種類別比單純數字更有外觀變異。

訓練損失、驗證分數與測試指標不能混用

- MLPRegressor 的 `score` 是 R^2 ; RMSE 需另算。
- MLPClassifier 的 `score` 是 accuracy ; log loss 另算。
- `loss_curve_` 記錄最佳化損失，可能包含正則化項。
- 早停監控的是內部驗證分數，不是最終測試結果。

教材案例提醒：高信心可能仍然答錯

教材 Fashion MNIST 約 90% accuracy 的模型，對不少錯誤仍給出接近 100% 的機率。

檢查錯誤案例與校準；一般框架可考慮 label smoothing。

scikit-learn 的 MLP 不直接提供完整的 label smoothing、dropout 或 TensorBoard 訓練介面。

調參次序：先讓模型學得動，再增加容量

檢查	可調項目	看什麼證據
資料尺度與切分	StandardScaler、資料角色	是否洩漏、分布是否不同
最佳化	學習率、batch size、optimizer	收斂曲線、警告、時間
容量	隱藏層數、每層神經元	訓練與驗證差距
正則化	alpha、早停	是否減少驗證誤差

深度、寬度與遷移學習

較深網路可重複組合中間特徵，某些函數可比淺層表示更有效率。

過窄的中間層可能形成資訊瓶頸；過多參數則增加訓練與資料需求。

遷移學習重用已有表示；本週先完成小型 MLP，後續再進入 PyTorch 與 GPU。

綜合實作：小型 MLP 與簡單模型比較

用已備妥的 digits，小型 MLP (32,) 與 (32,16) 各跑一次；與標準化 Logistic 比較。

提交：資料切分、accuracy、log loss、訓練時間、兩個錯誤案例。

離線組：10 輸入→50 隱藏→3 輸出的網路，列出各矩陣形狀並計算參數數目。

離堂檢核：把八週串回同一個學習流程

1. Backprop 計算什麼？Optimizer 改變什麼？
2. 二元、多類別、多標籤的輸出為何不同？
3. 為何深度模型仍需與簡單基準比較？

課後：Ch.9 練習 2-9；練習 1 的 playground 與練習 10 大型資料作延伸。